

Fundamentos conceptuales de la ciencia de datos

Alejandro Veloz

INF396 - Introducción a la Ciencia de Datos



Objetivos de la Unidad

1. Dar una **definición operativa** de ciencia de datos y ubicarla frente a enfoques estadísticos, ML, deep learning e IA.
2. Describir el aprendizaje como **minimización del riesgo** y el supuesto que comparte todo paradigma.
3. Distinguir algunos paradigmas de aprendizaje y problemas específicos.
4. Separar los tres problemas que se confunden bajo “ética de datos”: **equidad, privacidad, seguridad.**

Qué es la ciencia de datos

Problema real

Un hospital les entrega 180.000 fichas clínicas y una pregunta:

¿A qué pacientes conviene ofrecerles el programa de manejo crónico?

Problema real

Un hospital les entrega 180.000 fichas clínicas y una pregunta:

¿A qué pacientes conviene ofrecerles el programa de manejo crónico?

- ¿Es un problema de **estadística**, de **machine learning** o de **ingeniería de datos**?
- ¿Basta con predecir bien quién se va a enfermar?

Problema real

Un hospital les entrega 180.000 fichas clínicas y una pregunta:

¿A qué pacientes conviene ofrecerles el programa de manejo crónico?

- ¿Es un problema de **estadística**, de **machine learning** o de **ingeniería de datos**?
- ¿Basta con predecir bien quién se va a enfermar?

Esta pregunta se resolvió con un algoritmo usado sobre **70 millones** de personas al año (caso Obermeyer) - y estaba mal.

Tres tipos de definiciones

Tres tipos de definiciones

- **Por las herramientas:** “es Python, SQL, pandas y scikit-learn”. Confunde la disciplina con su stack.

Tres tipos de definiciones

- **Por las herramientas:** “es Python, SQL, pandas y scikit-learn”. Confunde la disciplina con su stack.
- **Por el tamaño:** “es estadística con datos grandes”. El tamaño cambia la ingeniería, no las preguntas.

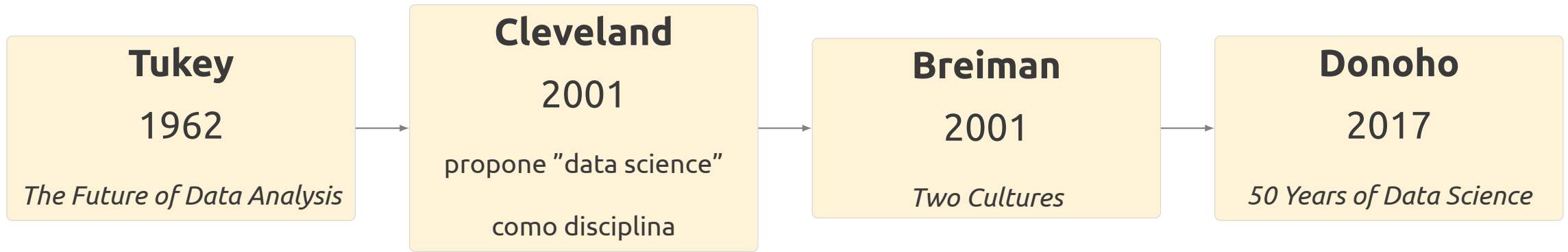
Tres tipos de definiciones

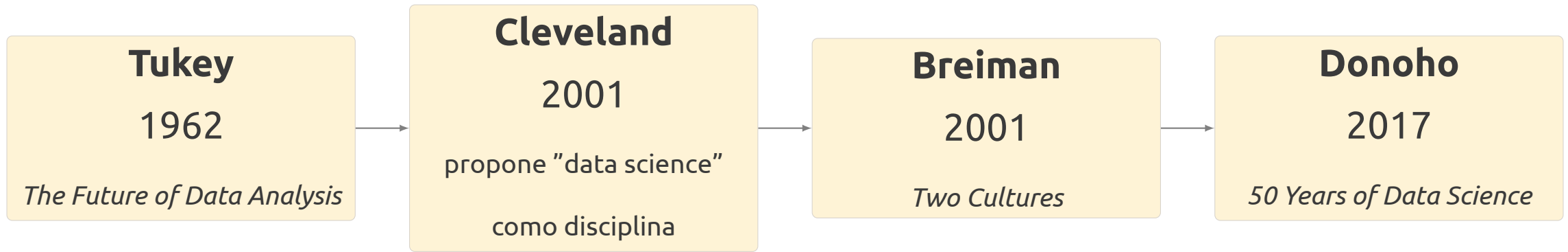
- **Por las herramientas:** “es Python, SQL, pandas y scikit-learn”. Confunde la disciplina con su stack.
- **Por el tamaño:** “es estadística con datos grandes”. El tamaño cambia la ingeniería, no las preguntas.
- **Por las disciplinas relacionadas:** “matemáticas + programación + dominio”. Describe a la persona, no al objeto de estudio.

Tres tipos de definiciones

- **Por las herramientas:** “es Python, SQL, pandas y scikit-learn”. Confunde la disciplina con su stack.
- **Por el tamaño:** “es estadística con datos grandes”. El tamaño cambia la ingeniería, no las preguntas.
- **Por las disciplinas relacionadas:** “matemáticas + programación + dominio”. Describe a la persona, no al objeto de estudio.

Ninguna dice *qué problema* resuelve la disciplina.





Tukey, en 1962, ya escribe que le interesa el **análisis de datos** y no la estadística matemática:

- Procedimientos para analizar datos.
- Las técnicas para interpretarlos.
- **Toda la maquinaria del análisis aplicada a datos reales.**

Breiman 2001: las dos culturas

Cultura del modelo de datos

La naturaleza genera los datos con un modelo conocido:

$$y = f_{\theta}(x) + \varepsilon$$

y se estima θ .

Objetivo: entender el mecanismo.

Validación: ajuste, residuos, significancia.

Cultura algorítmica

La naturaleza es una caja negra. Se busca $\hat{f}$ que prediga bien, sin comprometerse con su forma.

Objetivo: predecir.

Validación: error en datos no vistos.

Breiman 2001: las dos culturas

Cultura del modelo de datos

La naturaleza genera los datos con un modelo conocido:

$$y = f_{\theta}(x) + \varepsilon$$

y se estima θ .

Objetivo: entender el mecanismo.

Validación: ajuste, residuos, significancia.

Cultura algorítmica

La naturaleza es una caja negra. Se busca $\hat{f}$ que prediga bien, sin comprometerse con su forma.

Objetivo: predecir.

Validación: error en datos no vistos.

Breiman estimaba que la mayoría de los estadísticos vivía en la primera cultura. La ciencia de datos moderna es, en gran medida, la reconciliación de ambas.

Definición operativa

Definición operativa

La **ciencia de datos** es la disciplina que convierte datos en **decisiones defendibles**, cubriendo el ciclo completo: formular la pregunta, obtener y preparar los datos, modelar, cuantificar la incertidumbre, comunicar y evaluar las consecuencias de la decisión.

Definición operativa

Definición operativa

La **ciencia de datos** es la disciplina que convierte datos en **decisiones defendibles**, cubriendo el ciclo completo: formular la pregunta, obtener y preparar los datos, modelar, cuantificar la incertidumbre, comunicar y evaluar las consecuencias de la decisión.

- **Decisiones** - no “insights”. Si nadie actúa distinto, no hubo ciencia de datos.
- **Defendibles** - incertidumbre cuantificada y énfasis en los supuestos.
- **Ciclo completo** - el modelo es una etapa, no el objeto.

Donoho

1. Exploración y preparación de datos

2. Representación y transformación

3. Computación con datos

4. Modelamiento de datos

5. Visualización y presentación

6. Ciencia sobre la ciencia de datos

Donoho

1. Exploración y preparación de datos

2. Representación y transformación

3. Computación con datos

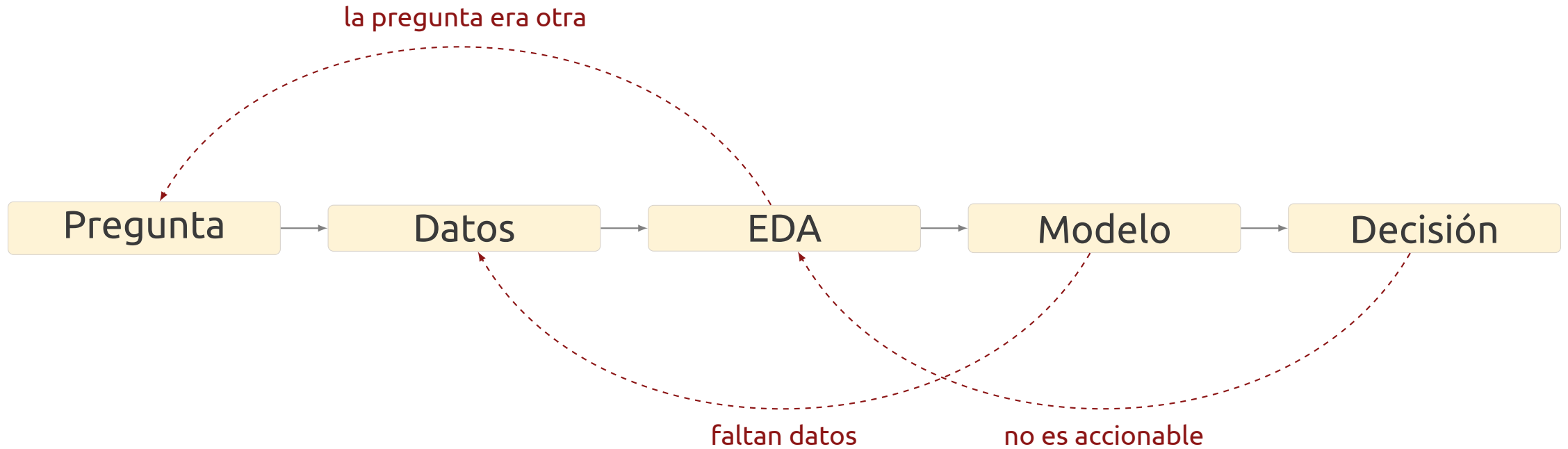
4. Modelamiento de datos

5. Visualización y presentación

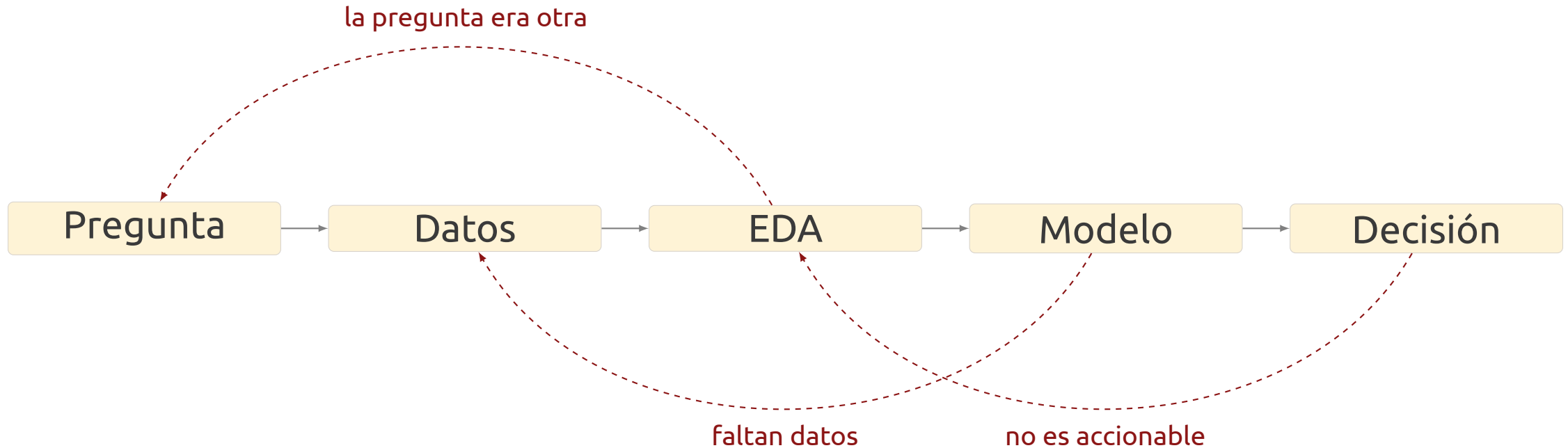
6. Ciencia sobre la ciencia de datos

Notar que **cinco de seis** no son “hacer modelos”. El bloque 4 es donde vive casi todo el ML; el 6 es el menos abordado: estudiar meta-científicamente qué métodos funcionan y por qué.

El ciclo de vida no es lineal



El ciclo de vida no es lineal



Las flechas rojas representan ciclos que emergen al enfrentar proyectos.

Un mito

“El 80 % del tiempo de un data scientist se va en limpiar datos”

Un mito

“El 80 % del tiempo de un data scientist se va en limpiar datos”

- La cifra viene de una nota del *New York Times* de **2014**, basada en entrevistas informales, no en un estudio.
- La encuesta *State of Data Science* de Anaconda (2020, n=2.360, 100+ países) mide **45 %** del tiempo en cargar y limpiar datos.

Qué **no** es ciencia de datos

- Correr 'model.fit()' sobre un CSV que alguien más armó.
- Reportar accuracy sin decir cuál es el costo de cada tipo de error.
- Encontrar una correlación y darle el peso de un hallazgo.
- Un dashboard que nadie mira antes de decidir.

Qué **no** es ciencia de datos

- Correr `'model.fit()'` sobre un CSV que alguien más armó.
- Reportar accuracy sin decir cuál es el costo de cada tipo de error.
- Encontrar una correlación y darle el peso de un hallazgo.
- Un dashboard que nadie mira antes de decidir.

El criterio: ¿alguien tomó una decisión distinta gracias a esto, y podríamos defenderla frente a alguien que la cuestione?

Intersecciones: estadística, ML, DL, IA

Dominios



Estadística vs machine learning

Modelo estadístico

- *¿Existe la relación, de qué tamaño?*
- Valida con IC, tests, residuos.
- **No necesita** conjunto de prueba.

Modelo de ML

- *¿Predigo bien lo que no vi?*
- Valida con error de generalización.
- El conjunto de prueba **es** el criterio.

Estadística vs machine learning

Modelo estadístico

- *¿Existe la relación, de qué tamaño?*
- Valida con IC, tests, residuos.
- **No necesita** conjunto de prueba.

Modelo de ML

- *¿Predigo bien lo que no vi?*
- Valida con error de generalización.
- El conjunto de prueba **es** el criterio.

Estadística y ML comparten cimientos, no objetivo.

```

n = 60
x = rng.uniform(0, 10, n)
y = 2.0 * x + rng.normal(0, 4.0, n)          # efecto verdadero: 2.0

X = np.c_[np.ones(n), x]                    # Cultura 1 – inferencia
beta = np.linalg.lstsq(X, y, rcond=None)[0]
res = y - X @ beta
se = np.sqrt((res @ res / (n-2)) * np.linalg.inv(X.T @ X)[1, 1])

xtr, xte, ytr, yte = train_test_split(x, y, test_size=.3, random_state=0)  # Cultura 2
mse = np.mean((LinearRegression().fit(xtr[:,None], ytr).predict(xte[:,None]) - yte)**2)

print(f"pendiente = {beta[1]:.2f}  IC95% = [{beta[1]-1.96*se:.2f}, {beta[1]+1.96*se:.2f}]")
print(f"MSE fuera de muestra = {mse:.2f}")

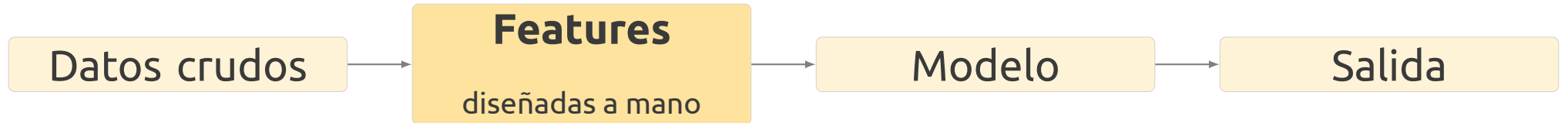
```

pendiente = 2.27 IC95% = [1.90, 2.64]

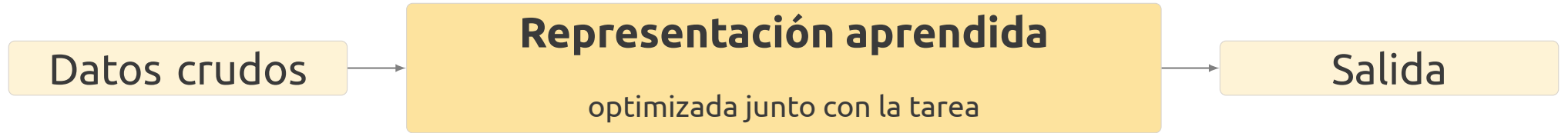
MSE fuera de muestra = 8.62

Deep learning

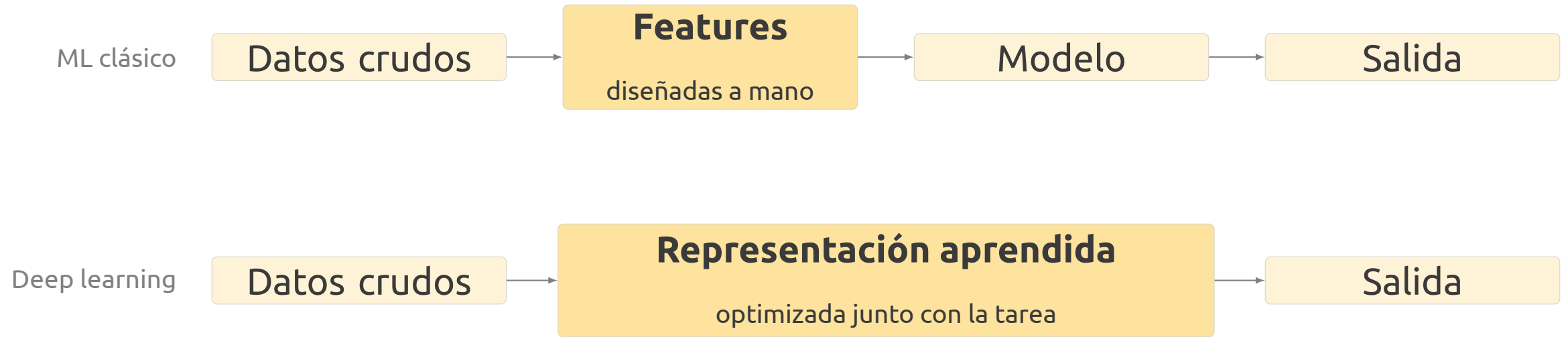
ML clásico



Deep learning



Deep learning



La ingeniería de características pasó a ser parte del problema de optimización.

El costo: se necesitan muchos más datos y se pierde interpretabilidad directa.

DS no es sólo el modelo



Sculley et al. (2015), *Hidden Technical Debt in Machine Learning Systems*.

	Pregunta central	Criterio de éxito
Estadística	¿Qué relación hay y cuán segura es?	Cobertura, calibración
Machine learning	¿Cómo predigo lo no visto?	Error de generalización
Deep learning	¿Cómo aprendo la representación?	Ídem, a mayor escala
Minería de datos	¿Qué patrones hay aquí dentro?	Interés, novedad
Data engineering	¿Cómo llegan los datos, confiables?	Latencia, calidad, costo
Ciencia de datos	¿Qué decisión tomo?	Valor y defendibilidad

Un proyecto de DS end-to-end

La pregunta y la decisión

Contexto: una cadena de retail con programa de fidelización y 30.000 clientes activos.

La pregunta y la decisión

Contexto: una cadena de retail con programa de fidelización y 30.000 clientes activos.

- **Decisión:** a cuáles **1.500** clientes enviar una oferta de retención (gift card de \$5.000), cada trimestre.
- **Quién decide:** la gerencia de marketing. El presupuesto es fijo.
- **Target:** *fuga* = el cliente no compra en los próximos 90 días.

La pregunta y la decisión

Contexto: una cadena de retail con programa de fidelización y 30.000 clientes activos.

- **Decisión:** a cuáles **1.500** clientes enviar una oferta de retención (gift card de \$5.000), cada trimestre.
- **Quién decide:** la gerencia de marketing. El presupuesto es fijo.
- **Target:** *fuga* = el cliente no compra en los próximos 90 días.

recurso limitado + priorización. ¿lo que importa es *quién fuga*, o *cuánto valor* se fuga? El target es una elección, no un dato.

Los datos: la base de clientes

En un proyecto real, esto sería la consulta al data warehouse. Por ahora **simulamos** la base de datos y así conocemos la verdad y podemos auditar cada paso:

```
rng_p = np.random.default_rng(7)                # generador del proyecto
n      = 30_000
antig  = rng_p.gamma(2.0, 18.0, n)                # meses en el programa
frec   = np.clip(rng_p.gamma(2.0, 6.0, n), 0.5, None) # compras por año
monto  = np.exp(rng_p.normal(10.3 + 0.015*frec, 0.45)) # gasto mensual (CLP)
rec    = np.minimum(rng_p.exponential(365/(frec+1)), 365) # días desde última compra
edad   = np.clip(rng_p.normal(46, 14, n), 18, 88)
app    = (rng_p.random(n) < expit(2.2 - 0.06*(edad-30))).astype(int) # usa la app
dist   = np.exp(rng_p.normal(1.2, 0.8, n))         # km a la tienda más cercana
```

El futuro que la tienda no ve

```
logit = (-0.45 + 0.010*np.minimum(rec,120) + 1.1*(rec>120) - 0.65*np.log1p(frec)
        - 0.8*app + 1.2*(app==0)*(rec>90) - 0.012*antig + 0.9*(antig<6)
        + 0.22*np.log1p(dist))
churn = (rng_p.random(n) < expit(logit)).astype(int)      # la verdad: quién fugará

edad_obs = np.where(rng_p.random(n) < np.where(app==1, 0.03, 0.55), np.nan, edad)

df = pd.DataFrame(dict(recencia=rec, frecuencia=frec, monto=monto, antigüedad=antig,
                        usa_app=app, distancia=dist, edad=edad_obs, churn=churn))
print(f"clientes: {n:},   tasa de fuga: {churn.mean():.1%}   edad faltante: {np.isnan(edad_obs).mean():.1%}")
```

clientes: 30,000 tasa de fuga: 15.0% edad faltante: 16.3%

El futuro que la tienda no ve

```
logit = (-0.45 + 0.010*np.minimum(rec,120) + 1.1*(rec>120) - 0.65*np.log1p(frec)
        - 0.8*app + 1.2*(app==0)*(rec>90) - 0.012*antig + 0.9*(antig<6)
        + 0.22*np.log1p(dist))
churn = (rng_p.random(n) < expit(logit)).astype(int)      # la verdad: quién fugará

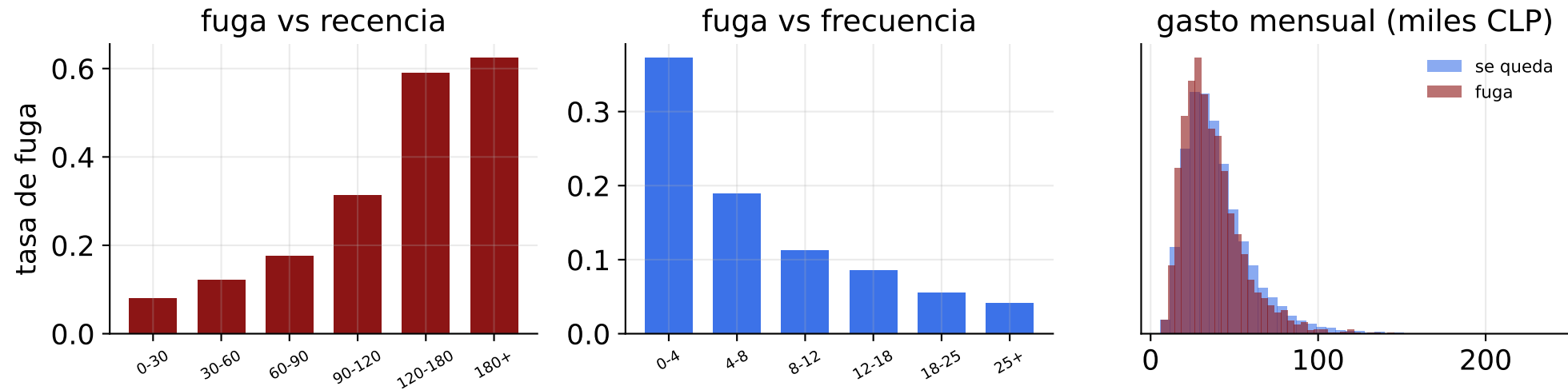
edad_obs = np.where(rng_p.random(n) < np.where(app==1, 0.03, 0.55), np.nan, edad)

df = pd.DataFrame(dict(recencia=rec, frecuencia=frec, monto=monto, antigüedad=antig,
                        usa_app=app, distancia=dist, edad=edad_obs, churn=churn))
print(f"clientes: {n:},   tasa de fuga: {churn.mean():.1%}   edad faltante: {np.isnan(edad_obs).mean():.1%}")
```

```
clientes: 30,000   tasa de fuga: 15.0%   edad faltante: 16.3%
```

El mecanismo real tiene **umbrales e interacciones** (recencia > 120 , cliente nuevo, sin app $\times$ inactivo) - y la edad falta **más** cuando el cliente no usa la app. Nada de esto se le dirá al modelo.

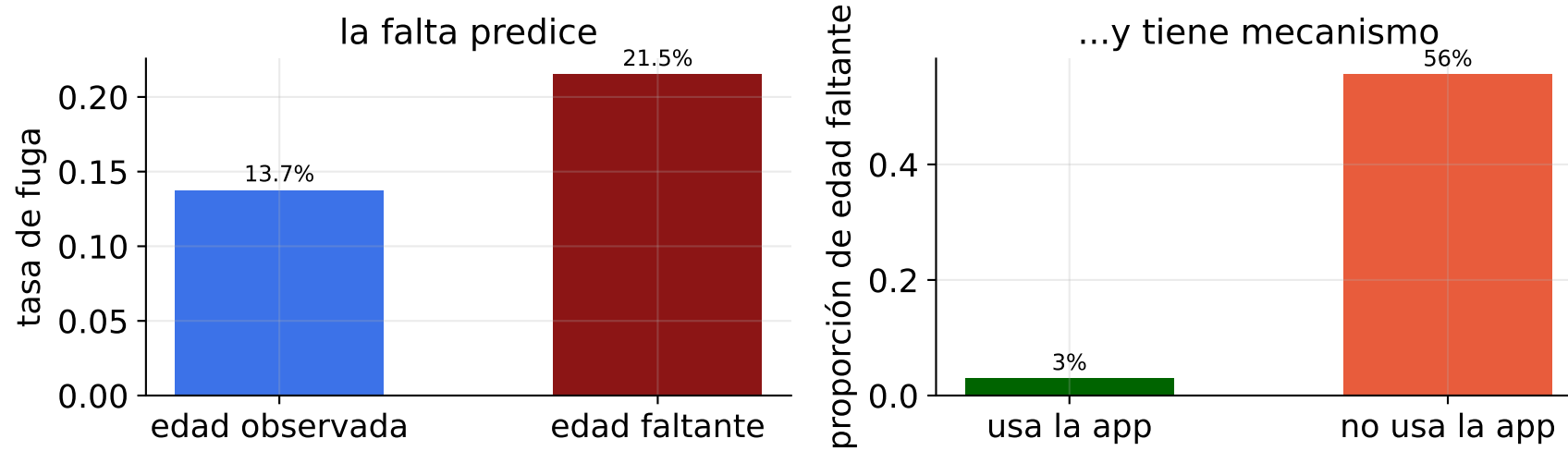
EDA: mirar antes de modelar



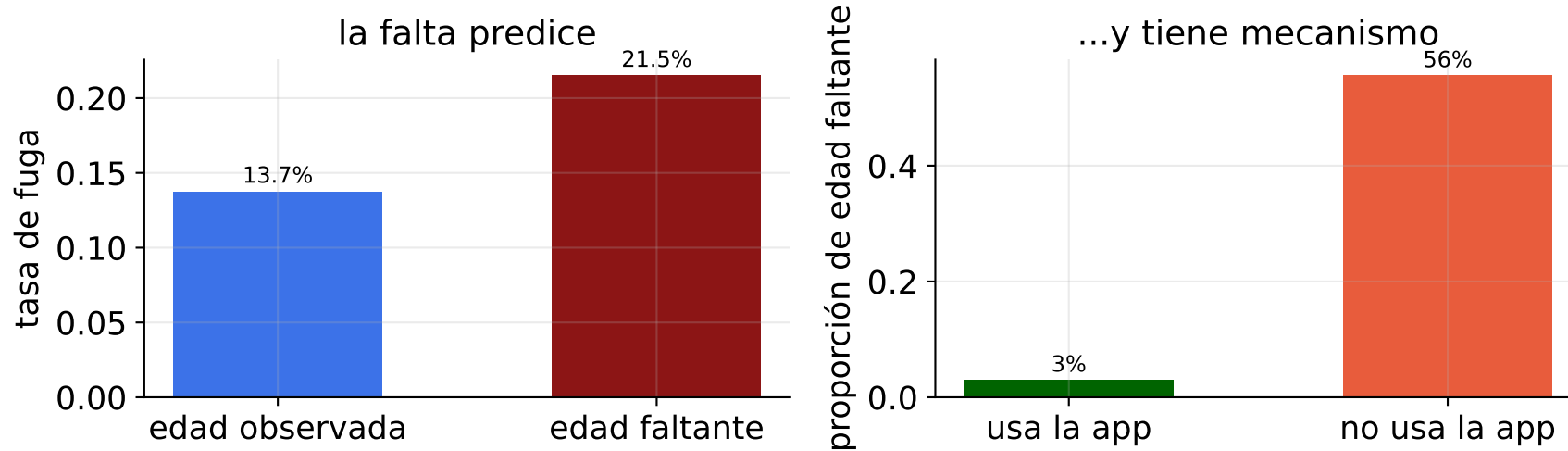
Recencia y frecuencia mandan; el gasto casi no distingue.

Esto ya nos adelanta que la heurística "contactar a los que más gastan" va a fallar.

Lo que falta también es dato



Lo que falta también es dato



La edad no falta al azar: falta cuando no hay app - y no tener app se asocia a fugar. **Borrar esas filas sesgaría la muestra**; imputar la media borraría la señal. Solución simple: conservar el *NaN* y usar un modelo que lo trate como categoría propia.

Modelos y validación

```
Xcols = ["recencia", "frecuencia", "monto", "antigüedad", "usa_app", "distancia", "edad"]
Xtr, Xte, ytr, yte = train_test_split(df[Xcols], df["churn"], test_size=0.3,
                                     stratify=df["churn"], random_state=0)

logi = make_pipeline(SimpleImputer(strategy="median"), StandardScaler(),
                    LogisticRegression(max_iter=1000)).fit(Xtr, ytr)
hgb = HistGradientBoostingClassifier(max_depth=3, learning_rate=0.08,
                                     max_iter=300, random_state=0).fit(Xtr, ytr)

for nombre, m in [("logística + imputación", logi), ("boosting, NaN nativo", hgb)]:
    p = m.predict_proba(Xte)[: , 1]
    print(f"{nombre:22s} AUC={roc_auc_score(yte, p):.3f} "
          f"PR-AUC={average_precision_score(yte, p):.3f} Brier={brier_score_loss(yte, p):.3f}")
```

logística + imputación AUC=0.801 PR-AUC=0.502 Brier=0.100

boosting, NaN nativo AUC=0.806 PR-AUC=0.554 Brier=0.095

Modelos y validación

```
Xcols = ["recencia", "frecuencia", "monto", "antigüedad", "usa_app", "distancia", "edad"]
Xtr, Xte, ytr, yte = train_test_split(df[Xcols], df["churn"], test_size=0.3,
                                      stratify=df["churn"], random_state=0)

logi = make_pipeline(SimpleImputer(strategy="median"), StandardScaler(),
                    LogisticRegression(max_iter=1000)).fit(Xtr, ytr)
hgb = HistGradientBoostingClassifier(max_depth=3, learning_rate=0.08,
                                     max_iter=300, random_state=0).fit(Xtr, ytr)

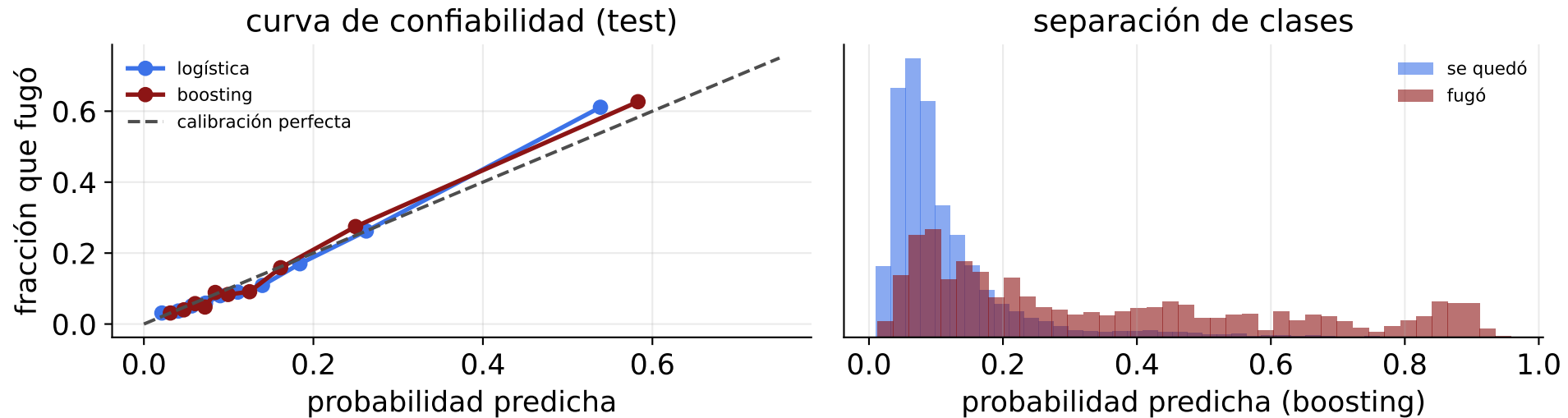
for nombre, m in [("logística + imputación", logi), ("boosting, NaN nativo", hgb)]:
    p = m.predict_proba(Xte)[: , 1]
    print(f"{nombre:22s} AUC={roc_auc_score(yte, p):.3f} "
          f"PR-AUC={average_precision_score(yte, p):.3f} Brier={brier_score_loss(yte, p):.3f}")
```

logística + imputación AUC=0.801 PR-AUC=0.502 Brier=0.100

boosting, NaN nativo AUC=0.806 PR-AUC=0.554 Brier=0.095

Test (9.000 clientes): la logística compite en AUC, el boosting gana en PR-AUC.

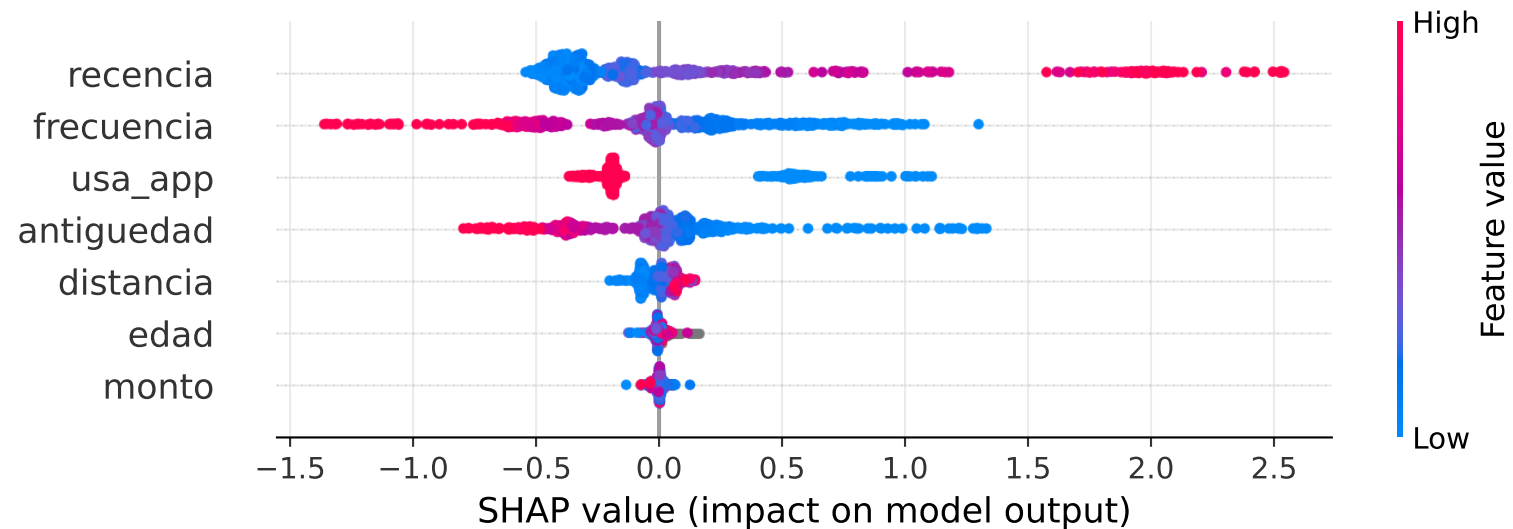
Calibración: que 30 % signifique 30 %



La decisión va a multiplicar estas probabilidades por dinero.

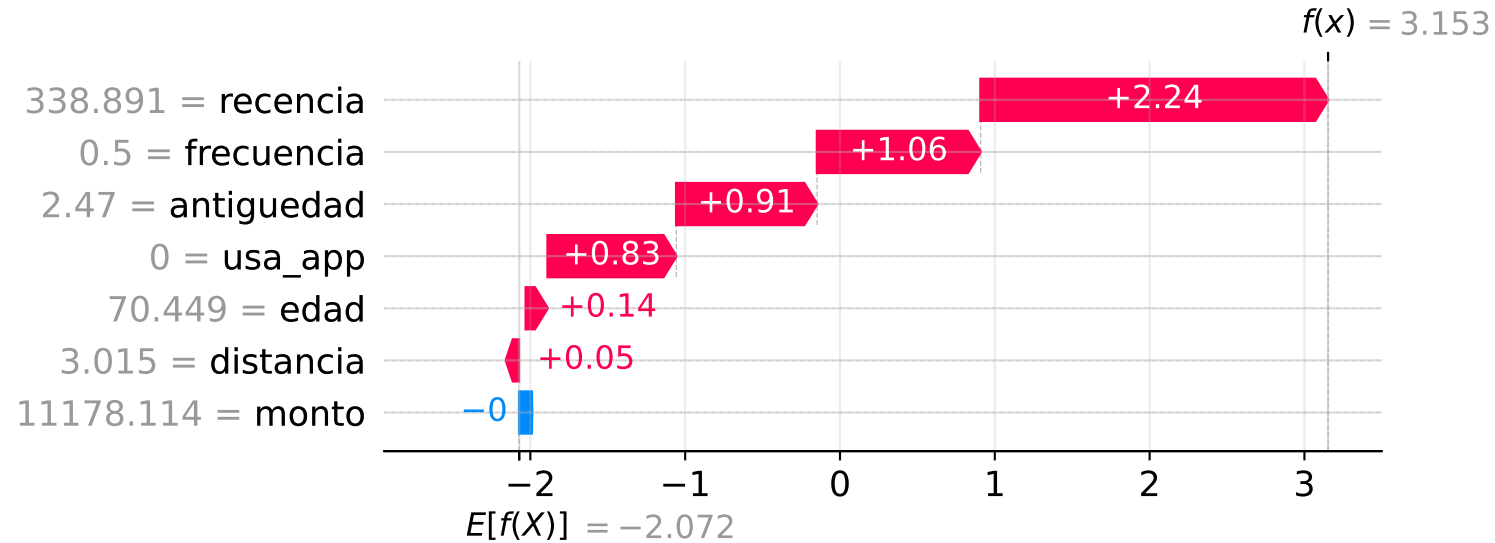
Un modelo que discrimina bien pero está mal calibrado ordena bien y
presupuesta mal.

Defendibilidad global: qué mueve al modelo

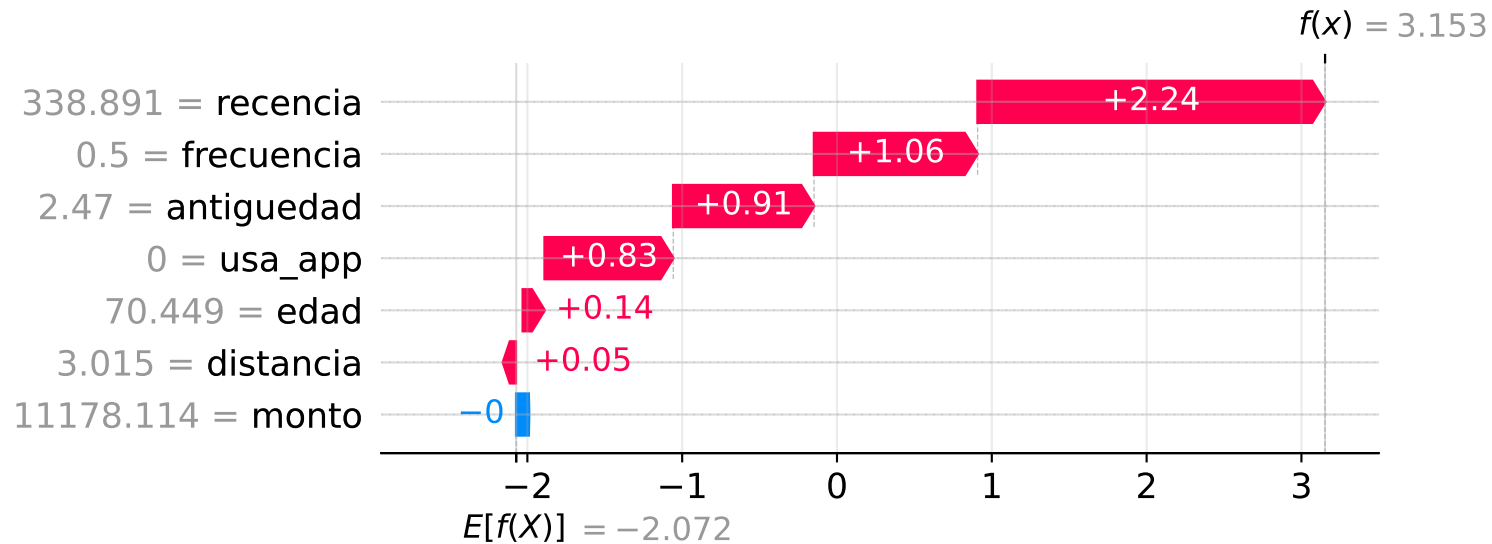


Valores SHAP (eje en log-odds): cuánto empuja cada variable la predicción de *este* cliente, hacia fuga (derecha) o permanencia (izquierda).

Defendibilidad local: por qué *este* cliente



Defendibilidad local: por qué *este* cliente



Esta es la lámina que se le muestra al gerente que pregunta “¿y por qué él?”. La respuesta deja de ser “porque el modelo lo dice”: cada peso está a la vista y se puede **discutir**.

Del puntaje al dinero

```
valor = 6 * Xte["monto"].values * 0.25      # valor en riesgo: 6 meses de margen
rho, costo = 0.30, 5000                     # efectividad histórica de la oferta y su costo
score = rho * p_hgb * valor                 # beneficio esperado de contactar a cada uno
util = rho * yte.values * valor - costo      # utilidad realizada (evaluación en test)

orden = np.argsort(-score)                  # ranking del modelo
o_gasto = np.argsort(-Xte["monto"].values)  # heurística de gerencia
perm = rng_p.permutation(util.size)         # control: al azar

print(f"top-1.500 por score del modelo : {util[orden][:1500].sum()/1e6:+.1f} M CLP")
print(f"top-1.500 por gasto           : {util[o_gasto][:1500].sum()/1e6:+.1f} M CLP")
print(f"1.500 al azar                 : {util[perm][:1500].sum()/1e6:+.1f} M CLP   (nadie: 0.0)")
```

```
top-1.500 por score del modelo : +4.9 M CLP
top-1.500 por gasto           : -1.8 M CLP
1.500 al azar                 : -4.1 M CLP   (nadie: 0.0)
```

Del puntaje al dinero

```
valor = 6 * Xte["monto"].values * 0.25      # valor en riesgo: 6 meses de margen
rho, costo = 0.30, 5000                     # efectividad histórica de la oferta y su costo
score = rho * p_hgb * valor                  # beneficio esperado de contactar a cada uno
util = rho * yte.values * valor - costo      # utilidad realizada (evaluación en test)

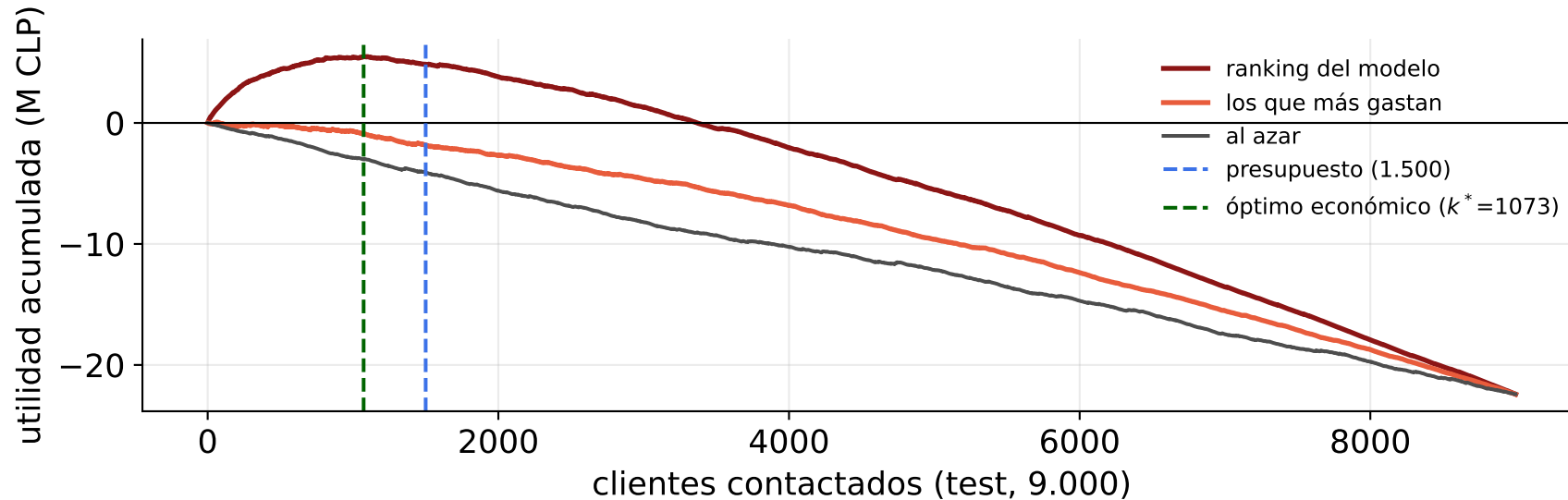
orden = np.argsort(-score)                   # ranking del modelo
o_gasto = np.argsort(-Xte["monto"].values)   # heurística de gerencia
perm = rng_p.permutation(util.size)          # control: al azar

print(f"top-1.500 por score del modelo : {util[orden][:1500].sum()/1e6:+.1f} M CLP")
print(f"top-1.500 por gasto : {util[o_gasto][:1500].sum()/1e6:+.1f} M CLP")
print(f"1.500 al azar : {util[perm][:1500].sum()/1e6:+.1f} M CLP (nadie: 0.0)")
```

```
top-1.500 por score del modelo : +4.9 M CLP
top-1.500 por gasto : -1.8 M CLP
1.500 al azar : -4.1 M CLP (nadie: 0.0)
```

El score no es p : es $\rho \cdot p \cdot \text{valor}$. La decisión ordena por dinero esperado, no por riesgo.

La curva de la decisión



La métrica final del proyecto no es el AUC: es esta curva.

Y el óptimo económico no tiene por qué coincidir con el presupuesto.